Welcome

Digital and Computer Vision

Lesson 2: Session 2

Thresholding , Edge Detection & Contours

Summary of the **previous** session: Smoothing Techniques

In the previous session we have covered :

Blurring and smoothing techniques (Average, Gaussian, Median, Bilateral filters).
Removing noise using image filtering.
Histogram equalization for contrast enhancement.

Summary of the current session: Thresholding , Edge Detection & Contours.

In this session we will covered the following:

Detecting edges using Sobel, Laplacian, and Canny edge detection.
Finding and drawing contours in images.
Tresholding, convex hulls and shape approximations.
Hough Transform

Exercise : from previous session

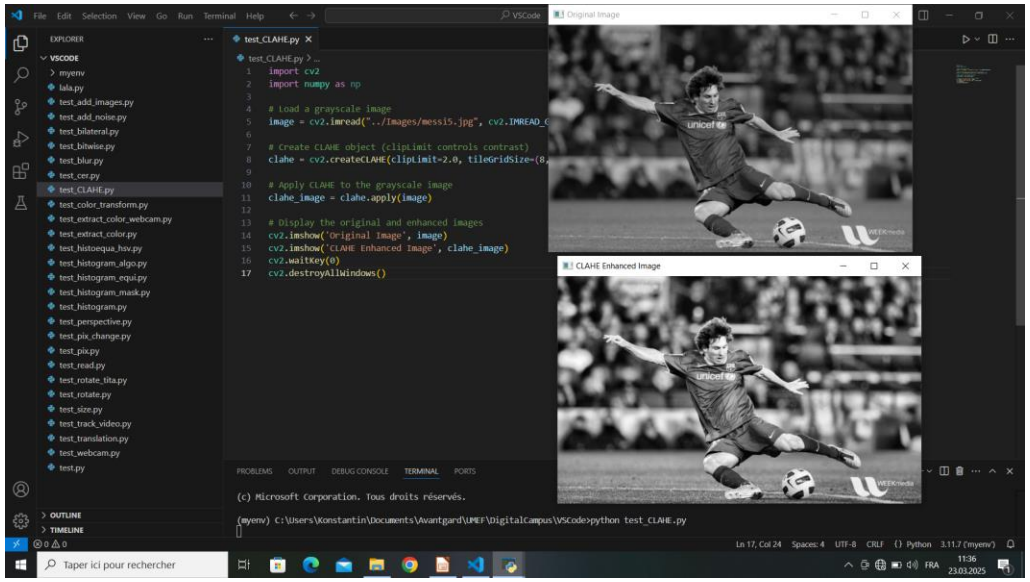
Apply and explain histogram equalization with the method CLAHE (Contrast Limited Adaptive Histogram Equalization) of OpenCV

Instead of applying the same histogram calculation to the whole image, CLAHE operates locally in small regions (tiles). Standard Histogram Equalization can cause over-bright areas and excessive noise in images with large intensity variations. CLAHE avoids this by limiting the contrast amplification.

Algorithm :
1. Divide Image into Small Regions (Tiles) : The image is split into small non-overlapping blocks (tiles), typically 8x8 or 16x16 pixels.
2. Apply Histogram Equalization to Each Tile : Histogram Equalization is applied separately to each tile.
3. Clip the Histogram (Limit Contrast Amplification) : A contrast limit (clip limit) is applied to avoid over-enhancing noise. Pixel intensities are redistributed to prevent over-saturation :
4. Smooth Transition Between Tiles : To avoid artificial edges, CLAHE blends neighboring tiles using bilinear interpolation.

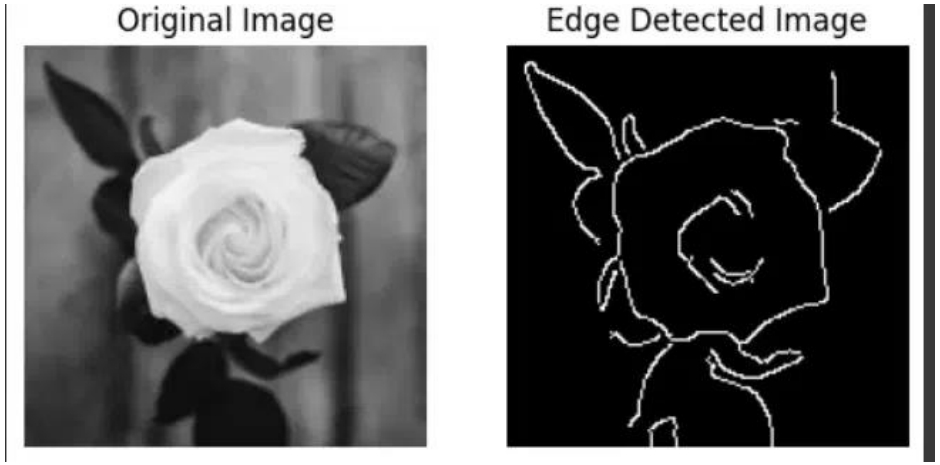
Exercise : from previous session code example

```
import cv2
import numpy as np

# Load a grayscale image
image = cv2.imread("../Images/messi5.jpg", cv2.IMREAD_GRAYSCALE)

# Create CLAHE object (clipLimit controls contrast)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

# Apply CLAHE to the grayscale image
clahe_image = clahe.apply(image)

# Display the original and enhanced images
cv2.imshow('Original Image', image)
cv2.imshow('CLAHE Enhanced Image', clahe_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Edges

Edges in an image are boundaries where intensity or color changes significantly, usually at object borders. These edges help identify structures and shapes, ideally forming a continuous outline around objects.



https://www.geeksforgeeks.org/what-is-edge-detection-in-image-processing/

Detecting edges using Sobel

Edge detection is used to identify object boundaries. The Sobel operator is a gradient-based edge detection method that calculates the rate of intensity change in an image. It applies two convolution kernels to detect edges in the X and Y directions separately.

1. Sobel Kernel for X-Direction (Detects Vertical Edges)

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

2. Sobel Kernel for Y-Direction (Detects Horizontal Edges)

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

To get the final edge map, we can combine both Sobel X and Sobel Y using Weighted Sum or Magnitude Formula:

$$G = \sqrt{G_x^2 + G_y^2}$$

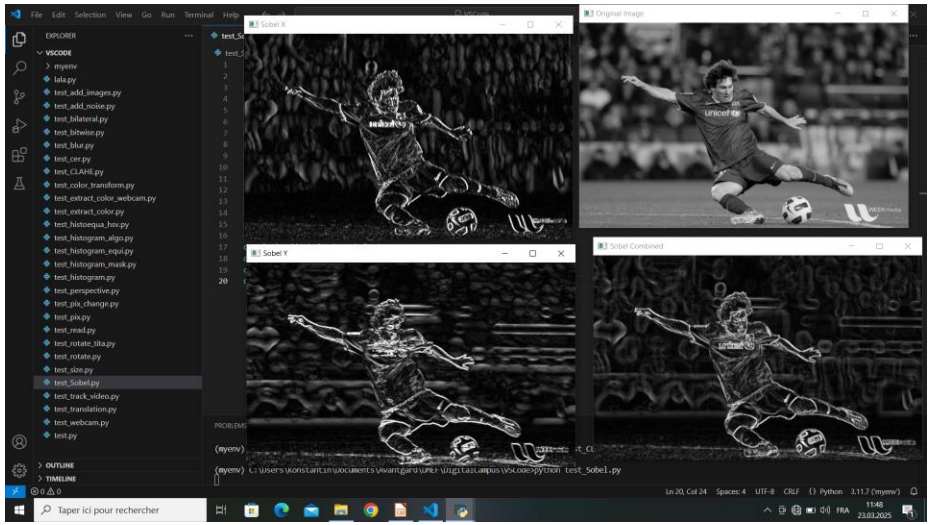
When to use Sobel edge detection ?

1. Best for detecting vertical and horizontal edges.
2. Less sensitive to noise than other gradient operations.
3. Works well for structured images (text, roads, objects).

Not the best for fine details—Canny edge detection may work better in some cases.

Sobel edge detection example example

```
import cv2

# Load an image
image = cv2.imread("../Images/messi5.jpg", cv2.IMREAD_GRAYSCALE)
# Apply Sobel filter in X direction (detects vertical edges)
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
# Apply Sobel filter in Y direction (detects horizontal edges)
sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
# Convert the gradients to absolute values and 8-bit
sobel_x = cv2.convertScaleAbs(sobel_x)
sobel_y = cv2.convertScaleAbs(sobel_y)
# Combine both X and Y gradients
sobel_combined = cv2.addWeighted(sobel_x, 0.5, sobel_y, 0.5, 0)
# Display the results
cv2.imshow('Original Image', image)
cv2.imshow('Sobel X', sobel_x)
cv2.imshow('Sobel Y', sobel_y)
cv2.imshow('Sobel Combined', sobel_combined)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



NB ! Black-to-White transition is taken as Positive slope (it has a positive value) while White-to-Black transition is taken as a Negative slope (It has negative value). So when you convert data to np.uint8, all negative slopes are made zero. In simple words, you miss that edge.

If you want to detect both edges, better option is to keep the output datatype to some higher forms, like cv.CV_16S, cv.CV_64F etc, take its absolute value and then convert back to cv.CV_8.

Canny edge detection

Canny Edge Detection is a multi-step algorithm developed by John F. Canny in 1986. It is one of the most widely used edge detection techniques because it provides clear and well-defined edges while reducing noise.

When to Use Canny Edge Detection?

1. Best for detecting well-defined edges
2. Works well for shape detection, segmentation, object tracking
3. Better than Sobel for real-world images with noise

Canny edge detection algorithm

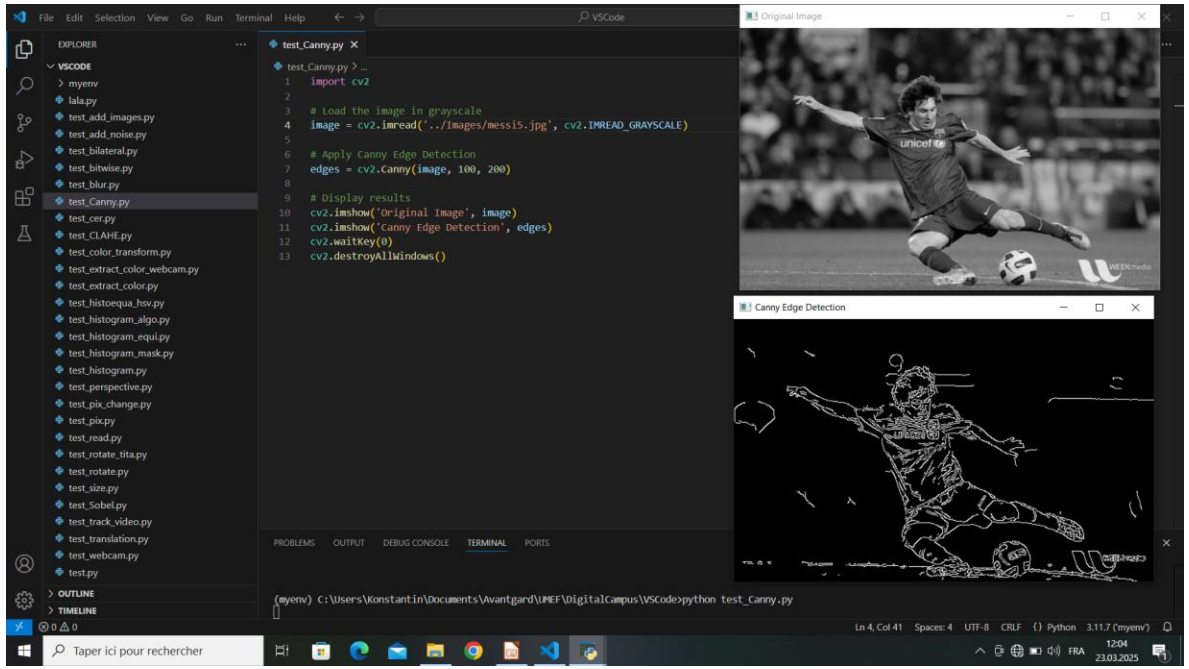
Canny Edge Detection follows a 4-step process:

1. **Noise Reduction (Gaussian Blur)** : The image is first smoothed using a Gaussian Blur to remove noise. This avoids detecting false edges caused by random intensity variations.
2. **Compute Gradient (Sobel Filter)**: Compute gradients in X and Y directions using Sobel operators. Detects regions with rapid intensity change (edges).
3. **Non-Maximum Suppression**: Every pixel is checked if it is a local maximum in its neighborhood in the direction of gradient. If so, it is considered for next stage, otherwise, it is suppressed (put to zero).
4. **Hysteresis Thresholding** : Classifies pixels into strong edges, weak edges, and non-edges. For this, we need two threshold values, minVal and maxVal.

Strong edges: above the maxVal ,Keep them in the final image.
Weak edges: between minVal and maxVal. Only kept if connected to strong edges.
Non-edges: below minVal - removed.

Canny edge detection example

```
import cv2

# Load the image in grayscale
image = cv2.imread('../Images/messi5.jpg', cv2.IMREAD_GRAYSCALE)

# Apply Canny Edge Detection
edges = cv2.Canny(image, 100, 200)

# Display results
cv2.imshow('Original Image', image)
cv2.imshow('Canny Edge Detection', edges)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Laplacian Edge Detection

Laplacian edge detection is a second-order derivative method used to detect edges in an image. Unlike Sobel or Canny, which use first-order derivatives (gradients), the Laplacian detects edges based on changes in the gradient.

Key Features:
1. Uses the Laplacian operator (∇^2) to detect edges.
2. Sensitive to noise because it uses second derivatives.
3. Highlights edges in all directions at once.

Laplacian Operator

The Laplacian operator (∇^2) is defined as:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

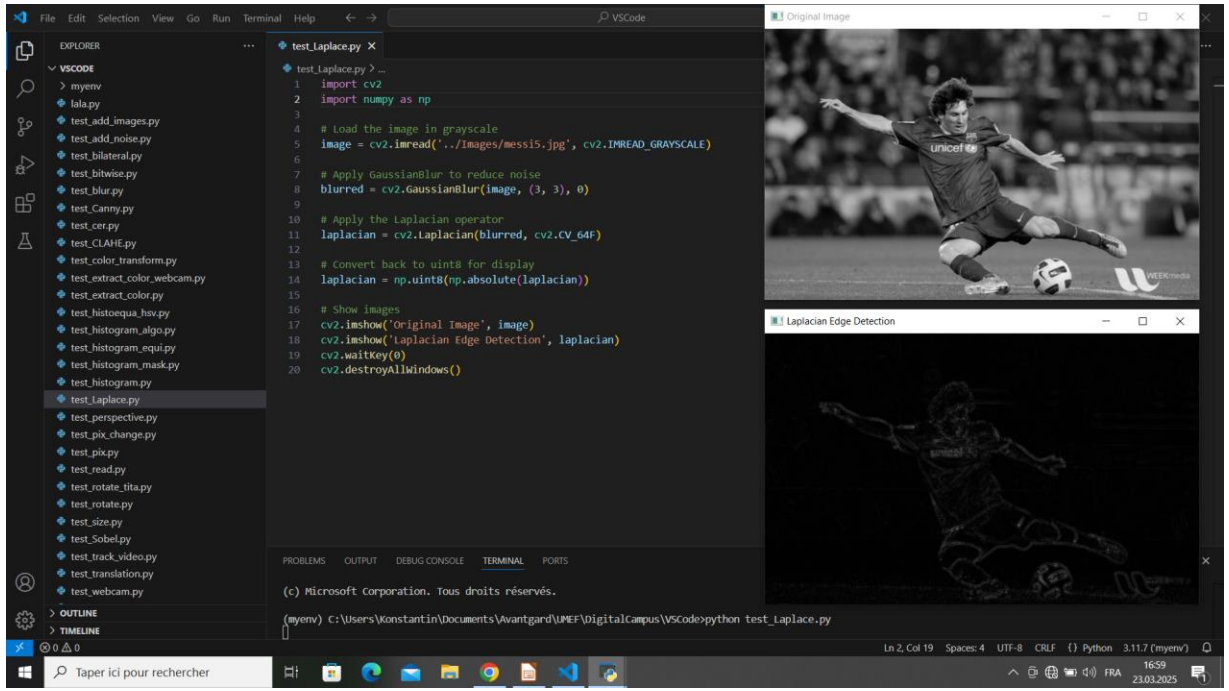
Where:

- $\frac{\partial^2 f}{\partial x^2}$ is the **second derivative** in the X-direction.
- $\frac{\partial^2 f}{\partial y^2}$ is the **second derivative** in the Y-direction.

Since it considers both directions, it detects **sharp changes in intensity** in all orientations.

Laplacian edge detection example

```
import cv2
import numpy as np

# Load the image in grayscale
image = cv2.imread('../Images/messi5.jpg', cv2.IMREAD_GRAYSCALE)
# Apply GaussianBlur to reduce noise
blurred = cv2.GaussianBlur(image, (3, 3), 0)
# Apply the Laplacian operator
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
# Convert back to uint8 for display
laplacian = np.uint8(np.absolute(laplacian))
# Show images
cv2.imshow('Original Image', image)
cv2.imshow('Laplacian Edge Detection', laplacian)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Summary edge detection techniques

Method	Type	Noise Sensitivity	Edge Thickness	Best Use Case
Sobel	First derivative	Moderate	Thick edges	Directional edges (horizontal/vertical)
Laplacian	Second derivative	High (needs blur)	Thin edges	Fast general edge detection
Canny	Multi-step	Low (uses noise reduction)	Sharp, clean edges	Precise and accurate edge detection

Tresholding

Thresholding is a technique used in image processing to convert a grayscale image into a binary image (black & white). It is useful for object segmentation, edge detection, and shape analysis.

In thresholding, each pixel is compared to a threshold value:
If the pixel value is greater than the threshold → it is set to white (255).
If the pixel value is less than the threshold → it is set to black (0).

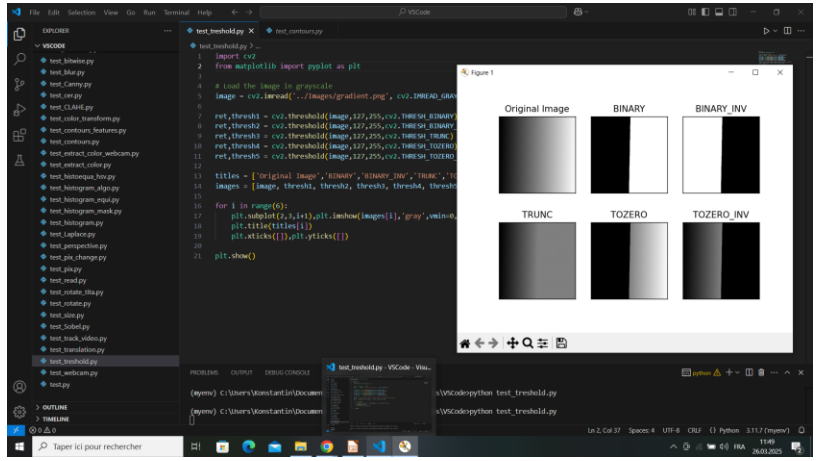
Simple Thresholding in OpenCV

Basic thresholding applies a fixed threshold to all pixels. The function cv.threshold is used to apply the thresholding. The first argument is the source image, which should be a grayscale image. The second argument is the threshold value which is used to classify the pixel values. The third argument is the maximum value which is assigned to pixel values exceeding the threshold. OpenCV provides different types of thresholding which is given by the fourth parameter of the function.

cv.THRESH_BINARY	white for pixel > treshold, else black
cv.THRESH_BINARY_INV	Inverted binary (black & white swapped)
cv.THRESH_TRUNC	Values > treshold set to treshold, rest unchanged
cv.THRESH_TOZERO	Values < treshold set to 0, rest unchanged
cv.THRESH_TOZERO_INV	Inverted ToZero
cv.THRESH_OTSU	Otsu's tresholding

Simple Thresholding example

```
import cv2
from matplotlib import pyplot as plt

# Load the image in grayscale
image = cv2.imread('../Images/gradient.png', cv2.IMREAD_GRAYSCALE)
ret,thresh1 = cv2.threshold(image,127,255,cv2.THRESH_BINARY)
ret,thresh2 = cv2.threshold(image,127,255,cv2.THRESH_BINARY_INV)
ret,thresh3 = cv2.threshold(image,127,255,cv2.THRESH_TRUNC)
ret,thresh4 = cv2.threshold(image,127,255,cv2.THRESH_TOZERO)
ret,thresh5 = cv2.threshold(image,127,255,cv2.THRESH_TOZERO_INV)
 titles = ['Original
Image','BINARY','BINARY_INV','TRUNC','TOZERO','TOZERO_INV']
images = [image, thresh1, thresh2, thresh3, thresh4, thresh5]
 for i in range(6):
    plt.subplot(2,3,i+1),plt.imshow(images[i],'gray',vmin=0,vmax=255)
    plt.title(titles[i])
    plt.xticks([]),plt.yticks([])
plt.show()
```



Adaptive Thresholding

If an image has uneven lighting, simple thresholding might not work well. In Adaptive thresholding the algorithm determines the threshold for a pixel based on a small region around it. So we get different thresholds for different regions of the same image which gives better results for images with varying illumination.

In addition to the parameters described above, the method cv.adaptiveThreshold takes three input parameters:

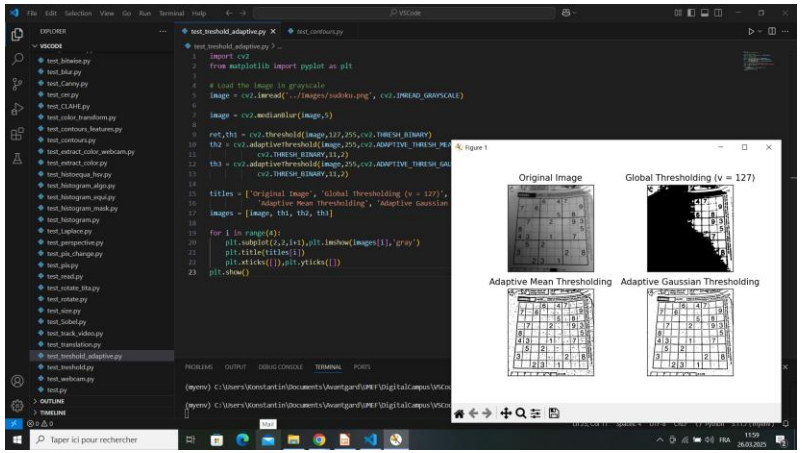
The adaptiveMethod decides how the threshold value is calculated:

cv.ADAPTIVE_THRESH_MEAN_C: The threshold value is the mean of the neighbourhood area minus the constant C.
cv.ADAPTIVE_THRESH_GAUSSIAN_C: The threshold value is a gaussian-weighted sum of the neighbourhood values minus the constant C.

The blockSize determines the size of the neighbourhood area and C is a constant that is subtracted from the mean or weighted sum of the neighbourhood pixels.

Adaptive Thresholding example

```
import cv2
from matplotlib import pyplot as plt

# Load the image in grayscale
image = cv2.imread('../Images/sudoku.png', cv2.IMREAD_GRAYSCALE)
image = cv2.medianBlur(image,5)
ret,th1 = cv2.threshold(image,127,255,cv2.THRESH_BINARY)
th2 = cv2.adaptiveThreshold(image,255,cv2.ADAPTIVE_THRESH_MEAN_C,\
            cv2.THRESH_BINARY,11,2)
th3 = cv2.adaptiveThreshold(image,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,\
            cv2.THRESH_BINARY,11,2)
 titles = ['Original Image', 'Global Thresholding (v = 127)',
            'Adaptive Mean Thresholding', 'Adaptive Gaussian Thresholding']
images = [image, th1, th2, th3]
 for i in range(4):
    plt.subplot(2,2,i+1),plt.imshow(images[i],'gray')
    plt.title(titles[i])
    plt.xticks([]),plt.yticks([])
plt.show()
```



Otsu's Thresholding

Otsu’s method determines the optimal threshold automatically. Otsu's method determines an optimal global threshold value from the image histogram. Otsu's algorithm split the image into two classes:
Pixels with values ≤ T (background).
Pixels with values > T (foreground).

then select the threshold value that minimizes the intra-class variance (or maximizes inter-class variance).
Otsu tresholding is effective for Bi-modal Histograms

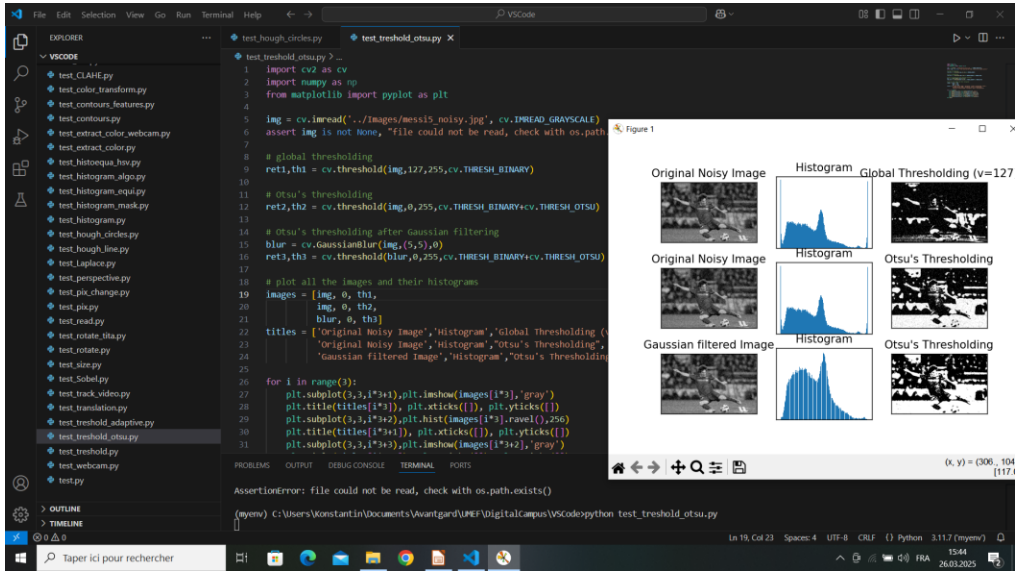
In OpenCV, the cv.threshold() function is used, where cv.THRESH_OTSU is passed as an extra flag. The threshold value can be chosen arbitrary. The algorithm then finds the optimal threshold value which is returned as the first output.

Otsu's Thresholding example 1/2

```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

img = cv.imread('../Images/messi5_noisy.jpg', cv.IMREAD_GRAYSCALE)

# global thresholding
ret1,th1 = cv.threshold(img,127,255,cv.THRESH_BINARY)

# Otsu's thresholding
ret2,th2 = cv.threshold(img,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)

# Otsu's thresholding after Gaussian filtering
blur = cv.GaussianBlur(img,(5,5),0)
ret3,th3 = cv.threshold(blur,0,255,cv.THRESH_BINARY+cv.THRESH_OTSU)
```

Otsu's Thresholding example 2/2

```
# plot all the images and their histograms
images = [img, 0, th1,
          img, 0, th2,
          blur, 0, th3]
titles = ['Original Noisy Image','Histogram','Global Thresholding (v=127)',
          'Original Noisy Image','Histogram',"Otsu's Thresholding",
          'Gaussian filtered Image','Histogram',"Otsu's Thresholding"]

for i in range(3):
    plt.subplot(3,3,i*3+1),plt.imshow(images[i*3],'gray')
    plt.title(titles[i*3]), plt.xticks([]), plt.yticks([])
    plt.subplot(3,3,i*3+2),plt.hist(images[i*3].ravel(),256)
    plt.title(titles[i*3+1]), plt.xticks([]), plt.yticks([])
    plt.subplot(3,3,i*3+3),plt.imshow(images[i*3+2],'gray')
    plt.title(titles[i*3+2]), plt.xticks([]), plt.yticks([])
plt.show()
```



Finding and drawing contours in images

Contours are curves that join continuous points along a boundary of objects with the same color or intensity in an image. They are useful in:

1. Object detection.
2. Shape analysis.
3. Image segmentation.
4. Measuring object properties (size, area, etc.).

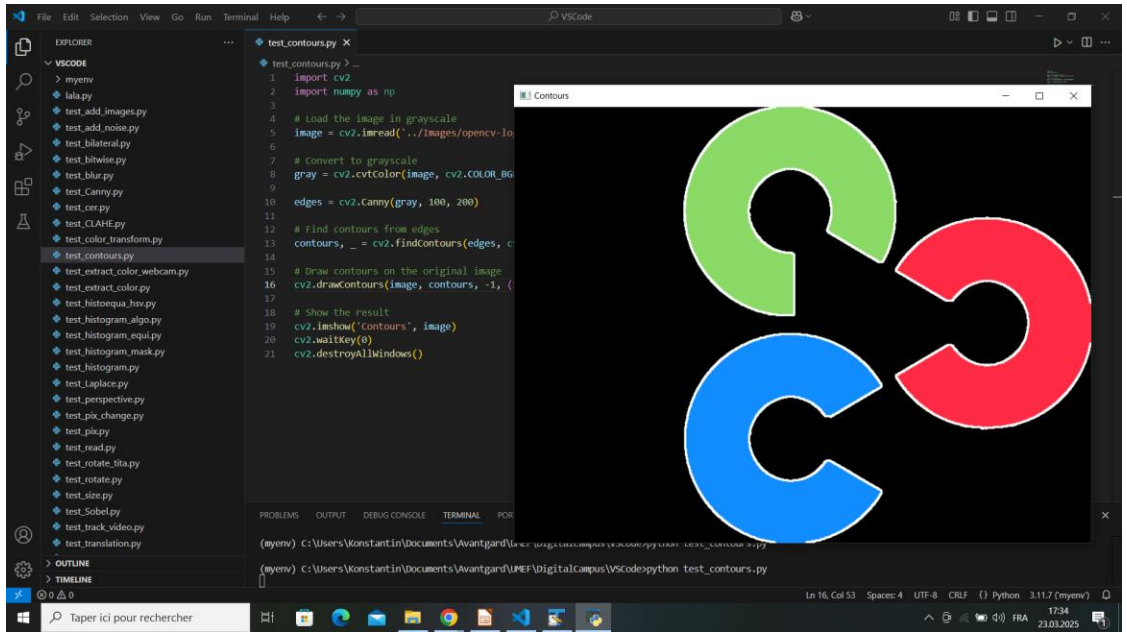
Since contours are computed only on binary images we have first convert the image to grayscale then apply thresholding or Canny edge detection.

Finding and drawing contours functions in OpenCV

To find contours :
contours, hierarchy = cv2.findContours(image, mode, method)
image → Binary image (must be thresholded or edge-detected).
mode → Defines contour retrieval:
cv2.RETR_EXTERNAL → Retrieves only the outermost contours.
cv2.RETR_TREE → Retrieves all contours in a hierarchical structure.
method → Contour approximation method:
cv2.CHAIN_APPROX_SIMPLE → Stores only important points, reducing memory usage.
cv2.CHAIN_APPROX_NONE → Stores all boundary points (more data, but unnecessary).

To draw contours :
cv2.drawContours(image, contours, contourIndex, color, thickness)
image → The image where contours are drawn.
contours → The list of contour points.
contourIndex → Index of the contour to draw (-1 draws all contours).
color → The color of the contour (e.g., (0, 255, 0) for green).
thickness → Line thickness (use -1 to fill the contour).

Finding and drawing contours example




```
import cv2

# Load the image
image = cv2.imread('../Images/opencv-logo.png')

# Convert to grayscale and find edges
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 100, 200)

# Find contours from edges
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

# Draw contours on the original image
cv2.drawContours(image, contours, -1, (255, 255, 255), 2)

# Show the result
cv2.imshow('Contours', image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Contour Features

Contours provide valuable shape and structure information about objects in an image shape classification (circles, rectangles, irregular shapes) and size meazurement. With OpenCV is posible to extract :

1. **Contour Area** : Measures the area enclosed by the contour.
2. **Contour Perimeter** : Measures the length of the contour boundary.
3. **Contour Centroid** :Finds the center (centroid) of a contour using image moments.
4. **Bounding Box** : Finds the smallest rectangle that encloses the contour.
5. **Minimum Enclosing Circle** : Finds the smallest circle enclosing the contour.
6. **Aspect Ratio (Width-to-Height Ratio)** : Measures how elongated an object is.
7. **Convex Hull** : Finds the smallest convex shape enclosing the contour.

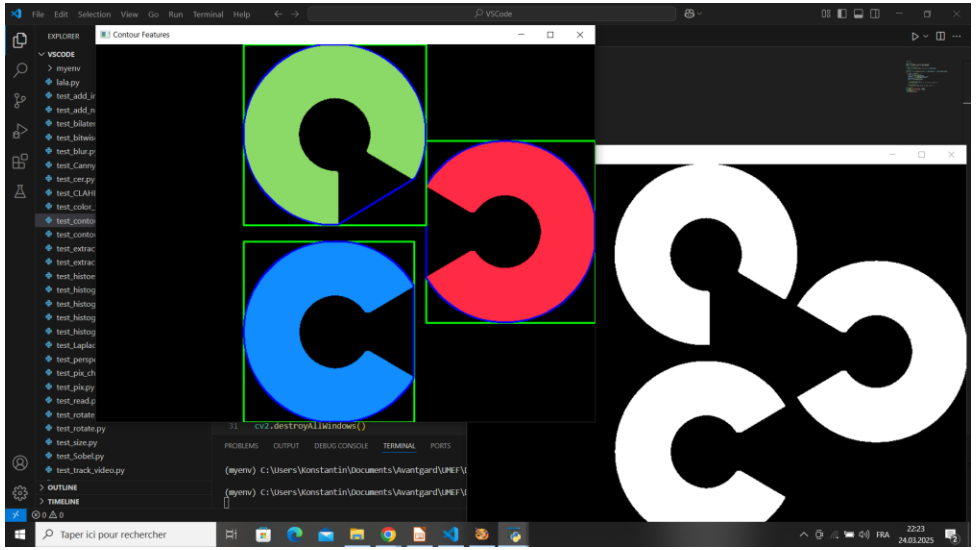
https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html

Contour Features example 1/2

```
import cv2

# Load image
image = cv2.imread('../Images/opencv-logo.png')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Apply thresholding
_, thresh = cv2.threshold(gray, 100, 255, cv2.THRESH_BINARY)

# Find contours
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

Contour Features example 2/2

```
for contour in contours:
    # Compute contour features
    area = cv2.contourArea(contour)
    perimeter = cv2.arcLength(contour, True)
    x, y, w, h = cv2.boundingRect(contour)
    aspect_ratio = float(w) / h
    hull = cv2.convexHull(contour)
    # Draw bounding rectangle
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
    # Draw convex hull
    cv2.drawContours(image, [hull], 0, (255, 0, 0), 2)

# Show results
cv2.imshow('Contour Features', image)
cv2.imshow('Tresholded Image', thresh)
cv2.waitKey(0)
cv2.destroyAllWindows()
cv2.destroyAllWindows()
```



Hough Transform

Hough Transform is a powerful technique used to detect geometric shapes, if you can represent that shape in a mathematical form, like lines and circles, even if they are incomplete or noisy.

Types of Hough Transform in OpenCV :
1. Hough Line Transform – Used to detect straight lines.
2. Hough Circle Transform – Used to detect circles.

Hough Line Transform – how it works ?

A straight line in the image space (Cartesian) can be represented as:

$$y=mx+c$$

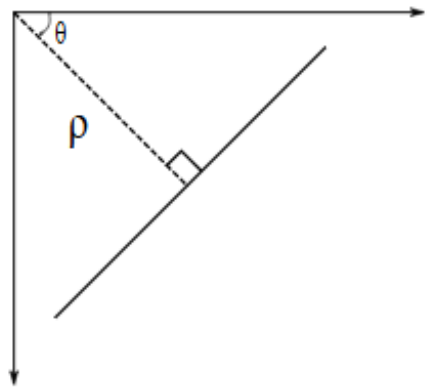
However, this equation has problems when m (slope) is infinite (vertical lines). So, we use an alternative parameterization:

$$\rho=x\cos\theta+y\sin\theta$$

Where:

ρ = perpendicular distance from the origin to the line

θ = angle between the x-axis and the perpendicular line



Key Idea:

Instead of detecting lines directly, we transform all edge points into the (ρ, θ) space, where they form a curve. Points that lie on the same line in image space will intersect at a common (ρ, θ) point in Hough space.

Hough Line Transform Algorithm

1. **Binarisation** : Input image should be a binary image, so apply threshold or use canny edge detection before applying hough transform
2. **Create a 2D Accumulator Matrix** : We create a 2D matrix called the Hough Accumulator where rows represent values of ρ, columns represent values of θ (from 0° to 180°). Each cell in this matrix is initialized to zero.
3. **Vote in the Hough Space** : For each edge point (x, y) in the image we calculate (ρ, θ). we increment the corresponding (ρ, θ) cell in the accumulator.
4. **Find Peaks in the Accumulator** : The accumulator now contains peaks where many edge points contribute to the same (ρ, θ). These peaks correspond to lines in the image.
5. **Convert (ρ, θ) Back to Cartesian** : After detecting peaks in the Hough space, we convert them back into lines. The detected (ρ, θ) values are converted back to line equations in Cartesian space and drawn on the image.

Hough Line Transform in OpenCV

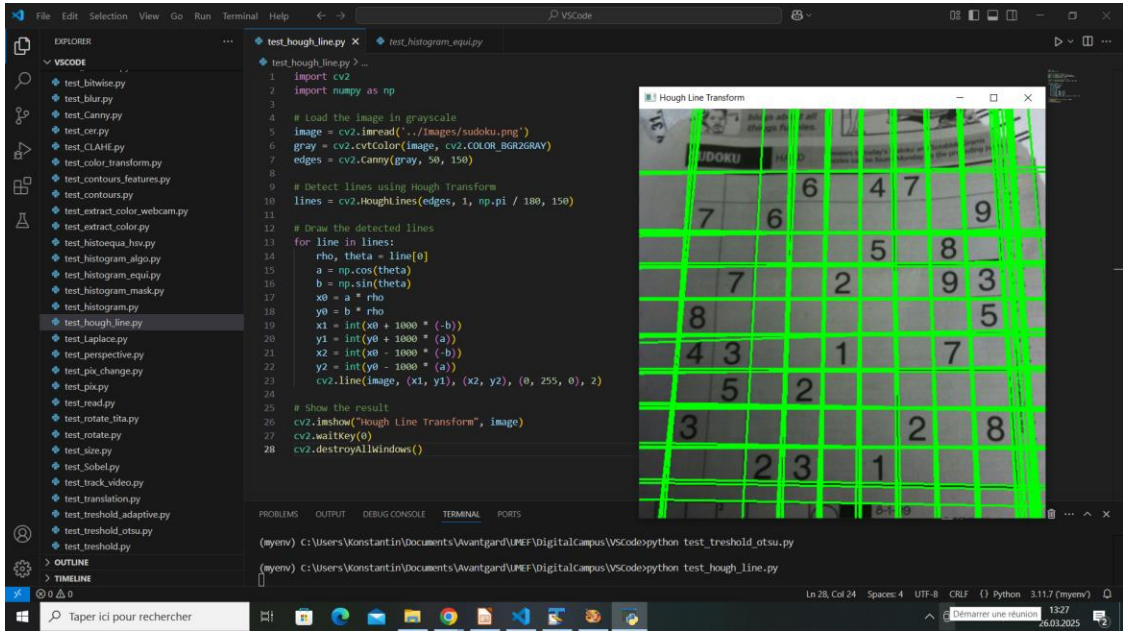
OpenCV's propose cv2.HoughLines() function with parameters
Input binarised image
Distance resolution of the accumulator (step size for ρ in pixels).
Angular resolution (step size for θ in radians).
Threshold (minimum votes needed to consider it a line).

Hough Line Transform example 1/2

```
import cv2
import numpy as np

# Load the image in grayscale
image = cv2.imread('../Images/sudoku.png')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

# Detect lines using Hough Transform
lines = cv2.HoughLines(edges, 1, np.pi / 180, 150)
```

Hough Line Transform example 2/2

```
# Draw the detected lines
for line in lines:
    rho, theta = line[0]
    a = np.cos(theta)
    b = np.sin(theta)
    x0 = a * rho
    y0 = b * rho
    x1 = int(x0 + 1000 * (-b))
    y1 = int(y0 + 1000 * (a))
    x2 = int(x0 - 1000 * (-b))
    y2 = int(y0 - 1000 * (a))
    cv2.line(image, (x1, y1), (x2, y2), (0, 255, 0), 2)

# Show the result
cv2.imshow("Hough Line Transform", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Hough Circle Transform

A circle can be described by:

$$(x-a)^2 + (y-b)^2 = r^2$$

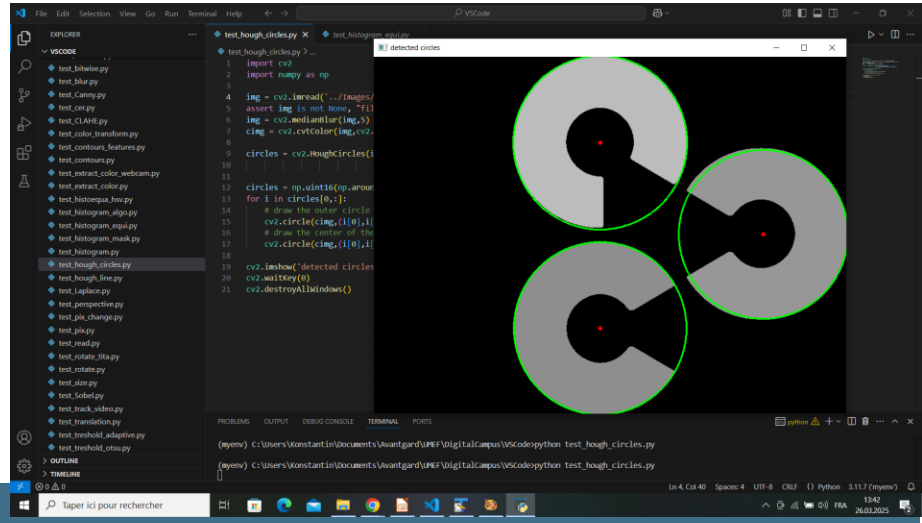
Where:

- (a, b) = center of the circle
- r = radius of the circle

Instead of detecting lines, we find circles by mapping points to a 3D accumulator space. From equation, we can see we have 3 parameters, so we need a 3D accumulator for hough transform, which would be highly ineffective. So OpenCV uses more trickier method, Hough Gradient Method which uses the gradient information of edges.

Hough Circle Transform example

```
import cv2
import numpy as np

img = cv2.imread('../Images/opencv-logo.png', cv2.IMREAD_GRAYSCALE)

cimg = cv2.cvtColor(img,cv2.COLOR_GRAY2BGR)
circles = cv2.HoughCircles(img,cv2.HOUGH_GRADIENT,1,20,
                           param1=50,param2=30,minRadius=0,maxRadius=0)
circles = np.uint16(np.around(circles))
for i in circles[0,:]:
    # draw the outer circle
    cv2.circle(cimg,(i[0],i[1]),i[2],(0,255,0),2)
    # draw the center of the circle
    cv2.circle(cimg,(i[0],i[1]),2,(0,0,255),3)

cv2.imshow('detected circles',cimg)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



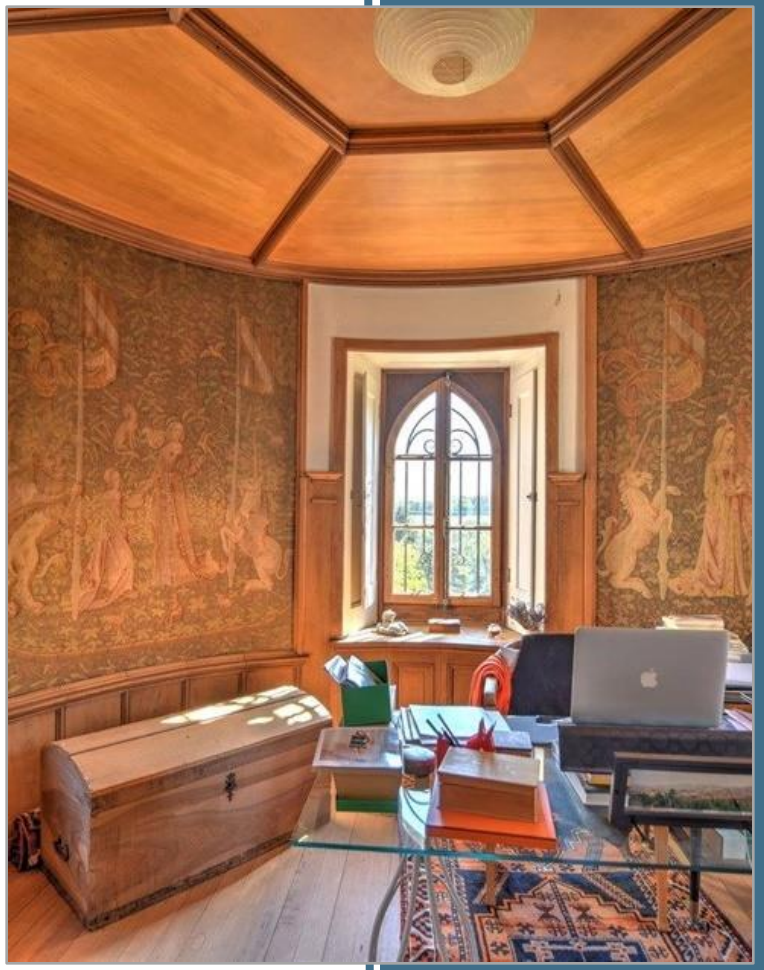
Exercise: Test contour detection

Test contour detection and drawing features on images rubberwhale1.png and stuff.jpg.
Use different thresholding methods and edge detection for binarization of the images.
Discuss the obtained results.

In this session we have covered:

Detecting edges using Sobel, Laplacian, and Canny edge detection.
Finding and drawing contours in images.
Tresholding, convex hulls and shape approximations.
Hough Transform

In the next session we will covered:

Image segmentation and object extraction
Watershed algorithm for image segmentation.
K-means clustering for color segmentation.
GrabCut for object extraction

THANKS !!!